

Vue.jsを用いたログイン画面の実装

・・・その前に

今回の構成では Thymeleafは使いません。

そのため各Controller.javaには、Thymeleafにより必要な値を送る設定は施しておりません。

なぜThymeleafを使わないのか？

従来のSpring Boot開発（Thymeleaf使用）と今回の開発（Vue.js使用）の違いは以下のようになります。

- **Thymeleaf（サーバーサイド・レンダリング）：**
サーバー側でHTMLにデータを埋め込んで、完成した「画面」をブラウザに送ります。
- **Vue.js + @RestController（クライアントサイド・レンダリング）：**
サーバー（Java）は「**生データ（JSON）**」だけを返します。画面をどう組み立てるかは、ブラウザ側で動く **Vue.js** が担当します。

Java側で値を `model.addAttribute()` する代わりに、Vue.js側が `axios` を使ってAPIを叩き、返ってきたJSONを受け取って画面に表示させる、という役割分担になっています。

ログインフォームの作成

それでは、最初の画面であるログインフォームを作りましょう。Vue.jsでは `.vue` というファイルに「見た目（HTML）」「動き（JS）」「装飾（CSS）」をまとめて書きます。

`frontend/src/components` フォルダに `LoginForm.vue` というファイルを新規作成し、以下のコードを記述してください。

```
<template>
  <div class="login-container">
    <h2>勤怠管理システム ログイン</h2>
    <form @submit.prevent="handleLogin">
      <div class="field">
        <label>ユーザーID (Email)</label>
        <input type="email" v-model="userId" required placeholder="example@example.com">
      </div>
      <div class="field">
        <label>パスワード</label>
        <input type="password" v-model="password" required>
      </div>
    </form>
  </div>
</template>
```

```

    <button type="submit">ログイン</button>
  </form>
  <p v-if="errorMessage" class="error">{{ errorMessage }}</p>
</div>
</template>

<script setup>
import { ref } from 'vue';
import apiClient from '../api'; // 先ほど作った設定ファイルを読み込む

const userId = ref('');
const password = ref('');
const errorMessage = ref('');

const handleLogin = async () => {
  try {
    // curlで叩いていたAPIをここで呼び出す
    const response = await apiClient.post('/users/login', {
      userId: userId.value,
      password: password.value
    });

    console.log('ログイン成功:', response.data);
    alert(`ようこそ、${response.data.userName}さん！`);
    // 本来はここで打刻画面へ遷移させますが、まずは疎通確認まで
  } catch (error) {
    console.error('ログイン失敗:', error);
    errorMessage.value = 'ユーザーIDまたはパスワードが正しくありません。';
  }
};
</script>

```

```
<style scoped>
.login-container {
  max-width: 400px;
  margin: 50px auto;
  padding: 20px;
  border: 1px solid #ddd;
  border-radius: 8px;
}
.field { margin-bottom: 15px; }
label { display: block; margin-bottom: 5px; }
input { width: 100%; padding: 8px; box-sizing: border-box; }
button { width: 100%; padding: 10px; background-color: #42b983; color: white; border: none; cursor: pointer; }
.error { color: red; margin-top: 10px; }
</style>
```

画面に表示させるための設定

作成した `LoginForm.vue` を実際にブラウザで見られるように、メインの `App.vue` を書き換えます。

`frontend/src/App.vue` の中身をすべて消して、以下に書き換えてください。

```
<template>
  <div id="app">
    <LoginForm />
  </div>
</template>

<script setup>
import LoginForm from './components/LoginForm.vue';
</script>
```

動作確認のポイント

1. バックエンドを起動 (`./mvnw spring-boot:run`)
2. フロントエンドを起動 (`npm run dev`)
3. ブラウザでログイン画面が表示されたら、以前 `curl` で登録したメールアドレスとパスワードを入力して「ログイン」ボタンを押してみてください。

「ようこそ、〇〇さん！」 とアラートが出れば、Vue.jsからJavaのAPIを呼び出すことに成功です！